

Linear Queue vs Circular Queue

Implementation-Based Performance Comparison and Analysis

Name	Deepak R
Register Number	192511008
Topic	Queue Data Structures

Question 1

Compare the performance of a linear queue and a circular queue when 100 enqueue and dequeue operations are performed alternately.

1.1 Understanding the Operation

In this question, an **enqueue** operation is followed immediately by a **dequeue** operation. This pair of operations is repeated 100 times. Therefore, a total of **100 enqueue + 100 dequeue = 200 queue operations** are performed for each implementation.

1.2 Linear Queue – Operation Flow

A linear queue uses two pointers: **front** points to the first element and **rear** points to the last element. During an alternating enqueue/dequeue sequence, an element is inserted and then immediately removed.

One alternating cycle:

```
Step 1: Enqueue A
[ A ] [   ] [   ] [   ] [   ]
  ↑
front/rear

Step 2: Dequeue A
[   ] [   ] [   ] [   ] [   ]
  ↑
front
```

The same pattern is repeated 100 times.

Because the newly inserted element is immediately removed, the queue does not continuously grow in size. Each enqueue and dequeue can be performed in **O(1)** time when front and rear are maintained as indices.

1.3 Circular Queue – Operation Flow

A circular queue also performs enqueue and dequeue in **O(1)** time, but its array is treated as circular. When the rear reaches the last position, it can return to index 0 using modulo arithmetic.

Example of wrap-around:

```
Before:
Index:  0    1    2    3    4
Queue: [ ] [ ] [C] [D] [E]
          ↑      ↑
        front  rear
```

```
Next rear:
(rear + 1) % SIZE
= (4 + 1) % 5
= 0
```

The next available position can be index 0.

For the exact alternating pattern in the question, the circular queue does not need to wait for wrap-around because the queue becomes empty after each dequeue. Nevertheless, its circular structure provides better

space reuse for other enqueue/dequeue patterns.

1.4 Performance Comparison for 100 Alternating Operations

Performance Factor	Linear Queue	Circular Queue
Enqueue operations	100	100
Dequeue operations	100	100
Total operations	200	200
Time per operation	O(1)	O(1)
Total time complexity	O(100) → O(n)	O(100) → O(n)
Space reuse	Limited	Efficient
Wrap-around	Not supported	Supported
False overflow	Possible	Avoided
For this exact workload	Very similar	Very similar

1.5 Result and Explanation

When 100 enqueue and dequeue operations are performed alternately, both queues perform the same number of operations: **100 enqueues and 100 dequeues**. Since both enqueue and dequeue are O(1), their basic execution-time complexity is the same.

Therefore, for this particular workload, there is **no significant performance difference in time** between the two queues. The major difference is memory utilization. The circular queue is more efficient because it can reuse positions that become free at the beginning of the array.

The circular queue becomes especially advantageous when multiple enqueue operations occur before dequeue operations. In such a case, a linear queue may reach the end of the array and report **Queue Full** even though positions at the beginning are empty. The circular queue avoids this problem by wrapping around.

Question 2

A queue implemented using an array reports “Queue Full” even though some positions are empty.

2.1 Analyze Why This Occurs

In a simple linear array queue, the rear pointer moves only forward. Consider an array of size 5. After inserting five elements, rear reaches index 4. If two elements are then removed, the first two positions become empty, but rear is still at index 4.

```
Initial:
Index:  0   1   2   3   4
Queue: [A] [B] [C] [D] [E]
front = 0, rear = 4
```

```
After two dequeues:
Index:  0   1   2   3   4
Queue: [ ] [ ] [C] [D] [E]
front = 2, rear = 4
```

```
Now an enqueue is attempted.
Linear queue checks:
rear == SIZE - 1
4 == 5 - 1  -> TRUE
```

It reports "Queue Full", although indexes 0 and 1 are empty.

This condition is called **false overflow**. The queue is logically not full, but the linear representation cannot move rear backward or wrap around to reuse the freed positions.

2.2 How a Circular Queue Solves the Problem

A circular queue treats the array as if the last position is connected to the first position. When rear reaches the last index, the next position is calculated using modulo arithmetic:

next rear = (rear + 1) % SIZE

```
Before reuse:
Index:   0   1   2   3   4
Queue:  [ ] [ ] [C] [D] [E]
front = 2, rear = 4
```

```
Next enqueue:
(rear + 1) % SIZE
= (4 + 1) % 5
= 0
```

So the new element is inserted at index 0.

```
After enqueue:
Index:   0   1   2   3   4
Queue:  [F] [ ] [C] [D] [E]
```

The circular queue therefore uses the empty positions at the beginning of the array instead of declaring the queue full. This provides better memory utilization without increasing the array size.

Overall Conclusion

For 100 alternating enqueue and dequeue operations, linear and circular queues have the same $O(1)$ time per operation and therefore similar time performance. The important advantage of the circular queue is **efficient space utilization**. A linear queue can suffer from false overflow because its rear pointer cannot wrap around, while a circular queue reuses freed positions through modulo arithmetic. Hence, circular queues are generally preferred for fixed-size array-based queues where insertion and deletion occur continuously.